JAVA PROGRAMMING
6-Month Practice Problem Workbook

400 structured exercises • 65–70 problems per month • Beginner to backend project

Alignment	Follows the uploaded 6-month Java roadmap, including weekly topics and project milestones.
Volume	65, 65, 70, 65, 70, and 65 problems by month: 400 total.
Suggested pace	About 3–4 problems per study day across six days each week; adjust for difficulty.
Method	Clarify input/output, write steps or pseudocode, code independently, test normal and edge cases, and record what you learned.
Solutions	Prompts only, so you can attempt them yourself. Ask for hints or solutions by problem number when needed.

Completion rule: A task is complete when it compiles, passes a normal test and a relevant edge case, and you can explain the logic without copying a solution.

Month 1 — Java Fundamentals

Days 1–28 | Variables, input/output, operators, decisions, loops, methods, arrays, strings, Student Grade Calculator

- **1.** Print your name, department, and college on separate lines.
- **2.** Print a welcome message using newline and tab escape sequences.
- **3.** Declare and display variables of each primitive type.
- **4.** Read a name and age with Scanner and print a formatted sentence.
- **5.** Read two integers and display sum, difference, product, quotient, and remainder.
- **6.** Convert Celsius to Fahrenheit.
- **7.** Convert centimeters into meters and remaining centimeters.
- **8.** Calculate rectangle area and perimeter.
- **9.** Calculate circle area and circumference using a constant for pi.
- **10.** Swap two numbers using a temporary variable.
- **11.** Swap two integers without a temporary variable.
- **12.** Read five subject marks and calculate total and average.
- **13.** Convert seconds into hours, minutes, and seconds.
- **14.** Calculate simple interest from principal, rate, and time.
- **15.** Show the result of casting a decimal to int and explain truncation.
- **16.** Check whether an integer is positive, negative, or zero.
- **17.** Check whether a number is even or odd.
- **18.** Find the larger of two numbers.
- **19.** Find the largest of three numbers.
- **20.** Check whether a year is a leap year.
- **21.** Check whether a character is a vowel or consonant.
- **22.** Assign a grade from marks; reject marks outside 0–100.
- **23.** Build a calculator using switch for +, -, *, and /.
- **24.** Check whether a given age meets a specified eligibility threshold.
- **25.** Check whether three side lengths can form a triangle.
- **26.** Classify a valid triangle as equilateral, isosceles, or scalene.
- **27.** Calculate electricity charges using slab rates you define and document.
- **28.** Find quadratic equation roots, handling all discriminant cases.
- **29.** Build a menu-driven unit converter using switch.
- **30.** Check whether three angles form a valid triangle.
- **31.** Print integers from 1 to N.
- **32.** Print even numbers from 1 to N.
- **33.** Find the sum from 1 to N.
- **34.** Print a multiplication table for a given number.
- **35.** Calculate factorial using a loop.
- **36.** Count digits in an integer, including zero.
- **37.** Reverse the digits of an integer.
- **38.** Check whether an integer is a palindrome.
- **39.** Check whether a number is prime.
- **40.** Print primes in an inclusive range.
- **41.** Find GCD of two positive integers.
- **42.** Find LCM using GCD.
- **43.** Calculate the sum of digits.

- **44.** Check whether a number is an Armstrong number.
- **45.** Print a right triangle and a number pattern.
- **46.** Write a method returning the maximum of two integers.
- **47.** Overload methods to calculate circle and rectangle areas.
- **48.** Write a method that checks primality.
- **49.** Read N array values and print them in reverse order.
- **50.** Find array sum and average.
- **51.** Find minimum and maximum array values.
- **52.** Implement linear search returning index or -1.
- **53.** Count even and odd array values.
- **54.** Copy an array without clone().
- **55.** Find the second-largest distinct value and handle its absence.
- **56.** Rotate an array right by one position.
- **57.** Merge two arrays into a third array.
- **58.** Add two compatible matrices.
- **59.** Find the transpose of a matrix.
- **60.** Count vowels, consonants, digits, and spaces in a string.
- **61.** Reverse a string without StringBuilder.reverse().
- **62.** Check string palindrome ignoring case.
- **63.** Count words while handling extra spaces.
- **64.** Count occurrences of a chosen character.
- **65.** Build a multi-student Grade Calculator with validation and summary.

Monthly total: 65 problems. Mark a problem complete only after testing and explaining your approach.

Month 2 — OOP and Intermediate Java

Days 29–56 | Classes, objects, constructors, OOP pillars, recursion, packages, and language features

- **1.** Create a Student class with fields, constructor, and display method.
- **2.** Create three Book objects and display their details.
- **3.** Create Rectangle with area and perimeter methods.
- **4.** Create Circle with a radius-initializing constructor.
- **5.** Create Employee and calculate annual salary.
- **6.** Use constructor overloading in a Product class.
- **7.** Use this to distinguish fields from parameters.
- **8.** Count created objects using a static field.
- **9.** Write a static utility method for temperature conversion.
- **10.** Create BankAccount with deposit and withdrawal.
- **11.** Reject negative deposits and withdrawals above balance.
- **12.** Create Mobile with brand, model, and price.
- **13.** Compare prices of two Laptop objects.
- **14.** Store five Student objects in an array and display them.
- **15.** Model a cart item with quantity and unit price.
- **16.** Encapsulate Person fields with validated setters.
- **17.** Create Person and Student using inheritance.
- **18.** Create Employee and Manager with overridden display methods.
- **19.** Create Vehicle with Car and Bike subclasses.
- **20.** Use super to invoke a parent constructor.
- **21.** Create abstract Shape with abstract area method.
- **22.** Implement Circle and Rectangle subclasses.
- **23.** Create Payment interface with Card and UPI simulation classes.
- **24.** Demonstrate runtime polymorphism using Shape references.
- **25.** Overload calculate for int, double, and three operands.
- **26.** Implement two interface methods in a class.
- **27.** Create a final class and test extension restriction.
- **28.** Demonstrate public, protected, and package-private access across packages.
- **29.** Model Student has Courses using composition.
- **30.** Compare two strings using == and equals; explain results.
- **31.** Count repeated characters using nested loops.
- **32.** Remove spaces from a string.
- **33.** Replace vowels with asterisks.
- **34.** Use StringBuilder to build a comma-separated list.
- **35.** Reverse each word while preserving word order.
- **36.** Find the largest row sum in a 2D array.
- **37.** Find main diagonal sum of a square matrix.
- **38.** Multiply compatible matrices.
- **39.** Create a jagged array for different mark counts.
- **40.** Write recursive factorial with a base case.
- **41.** Write recursive sum from 1 to N.
- **42.** Write recursive Fibonacci and note its performance.
- **43.** Write recursive digit sum.

- ■ **44.** Write recursive binary search on a sorted array.
- ■ **45.** Create a final application-name constant.
- ■ **46.** Create an OrderStatus enum and print its values.
- ■ **47.** Write a varargs method that sums integers.
- ■ **48.** Read command-line arguments and print indexed values.
- ■ **49.** Create generic Box with setter and getter.
- ■ **50.** Write a generic method to print an array.
- ■ **51.** Start a Student Management console menu.
- ■ **52.** Add a student with ID, name, and marks.
- ■ **53.** List all students in a readable format.
- ■ **54.** Search a student by ID.
- ■ **55.** Update marks with validation.
- ■ **56.** Delete a student by ID.
- ■ **57.** Prevent duplicate student IDs.
- ■ **58.** Calculate student averages and grades.
- ■ **59.** Display the student with highest average.
- ■ **60.** Count students by grade category.
- ■ **61.** Sort students by average using Comparator.
- ■ **62.** Handle invalid menu choices without exiting.
- ■ **63.** Separate Student, StudentService, and Main classes.
- ■ **64.** Write a README with features, compile steps, and sample usage.
- ■ **65.** Create a small menu-driven calculator class using methods and constructor initialization.

Monthly total: 65 problems. Mark a problem complete only after testing and explaining your approach.

Month 3 — Collections and Modern Core Java

Days 57–84 | Exceptions, Collections, Generics, sorting, lambdas, streams, Optional, date/time

- **1.** Handle division by zero with a clear message.
- **2.** Handle invalid integer input from Scanner.
- **3.** Handle an invalid array index safely.
- **4.** Use multiple catch blocks for distinct failures.
- **5.** Demonstrate finally for cleanup.
- **6.** Throw IllegalArgumentException for invalid age.
- **7.** Create InsufficientBalanceException.
- **8.** Create InvalidMarkException for marks outside 0–100.
- **9.** Declare throws and handle the exception at the caller.
- **10.** Show checked and unchecked exception examples.
- **11.** Validate usernames with a custom exception.
- **12.** Keep a menu running after recoverable input errors.
- **13.** Store ten names in ArrayList and print them.
- **14.** Insert a name at a specified list position.
- **15.** Remove matching values from a list.
- **16.** Find the index of a target in a list.
- **17.** Remove duplicates while preserving insertion order.
- **18.** Store unique email addresses in a Set.
- **19.** Compare HashSet and TreeSet iteration behavior.
- **20.** Simulate a service queue using Queue.
- **21.** Use Deque at both ends.
- **22.** Process tasks by priority using PriorityQueue.
- **23.** Build a contact directory using HashMap.
- **24.** Count word frequencies with HashMap.
- **25.** Find the key with the highest integer value.
- **26.** Group names by first letter using a map of lists.
- **27.** Sort integers ascending and descending.
- **28.** Iterate through a map using entrySet.
- **29.** Remove selected collection values safely with Iterator.
- **30.** Build undo history using Deque.
- **31.** Represent inventory as product ID to quantity.
- **32.** Compare ArrayList and LinkedList for specified workloads.
- **33.** Implement generic Pair.
- **34.** Write a generic list-element swap method.
- **35.** Write a bounded generic sum for Number values.
- **36.** Write a wildcard method that prints unknown-type lists.
- **37.** Implement Comparable by ID.
- **38.** Sort Student objects by marks using Comparator.
- **39.** Sort employees by salary then name.
- **40.** Sort strings by length using a lambda.
- **41.** Filter even numbers using a lambda.
- **42.** Create a two-number functional interface.
- **43.** Implement addition and multiplication with that interface.

- ■ **44.** Use a method reference to print list elements.
- ■ **45.** Use removeIf to delete values meeting a condition.
- ■ **46.** Print map entries with Map.forEach and lambda.
- ■ **47.** Create a generic in-memory repository holder.
- ■ **48.** Comment how generics improve type safety.
- ■ **49.** Filter marks above a threshold using streams.
- ■ **50.** Map names to uppercase.
- ■ **51.** Remove duplicates using distinct.
- ■ **52.** Sort products by price using streams.
- ■ **53.** Sum a list using reduce.
- ■ **54.** Collect names with a given prefix.
- ■ **55.** Count words longer than a threshold.
- ■ **56.** Find maximum salary with a stream.
- ■ **57.** Group employees by department.
- ■ **58.** Use Optional with a fallback for missing lookup.
- ■ **59.** Parse LocalDate and display date components.
- ■ **60.** Calculate days between two dates.
- ■ **61.** Create Employee data with department and salary.
- ■ **62.** Build Employee Management menu using ArrayList.
- ■ **63.** Search employee ID and handle missing records.
- ■ **64.** Filter employees by department and salary.
- ■ **65.** Sort by salary and display top three.
- ■ **66.** Generate department-wise employee counts.
- ■ **67.** Validate employee data with custom exceptions.
- ■ **68.** Document and test the in-memory employee manager.
- ■ **69.** Use streams to calculate the average of a list of numeric values.
- ■ **70.** Create a LocalDate-based reminder check that identifies dates before today.

Monthly total: 70 problems. Mark a problem complete only after testing and explaining your approach.

Month 4 — Advanced Core Java and JDBC

Days 85–112 | File I/O, concurrency, JVM, annotations, reflection, networking, JDBC, Library Management

- **1.** Create a directory and text file if absent.
- **2.** Write five lines using FileWriter.
- **3.** Read a file line by line using BufferedReader.
- **4.** Append a timestamped log line.
- **5.** Count lines, words, and characters in a file.
- **6.** Copy a text file using character streams.
- **7.** Copy a binary file using byte streams.
- **8.** List directory files filtered by extension.
- **9.** Rename a file after checking destination conflict.
- **10.** Serialize a Student object.
- **11.** Deserialize and display a Student object.
- **12.** Handle missing files and I/O errors.
- **13.** Create a thread by extending Thread.
- **14.** Create a thread using Runnable.
- **15.** Run two tasks and observe interleaving.
- **16.** Use sleep in a countdown thread.
- **17.** Use join so main waits for a worker.
- **18.** Handle interruption in a sleeping thread.
- **19.** Demonstrate a race condition on shared counter.
- **20.** Protect the counter with synchronized.
- **21.** Create producer-consumer coordination using wait/notifyAll.
- **22.** Submit tasks to ExecutorService.
- **23.** Collect Callable results using Future.
- **24.** Shut down an executor correctly.
- **25.** Explain thread safety and concurrency use cases.
- **26.** Print JVM available and maximum memory.
- **27.** Describe class loading and bytecode execution flow.
- **28.** Explain when objects become eligible for collection.
- **29.** Create and use a custom annotation.
- **30.** Use reflection to print declared fields and methods.
- **31.** Parse a URL using URI or URL.
- **32.** Resolve a host name using InetAddress.
- **33.** Write a TCP server that accepts one client.
- **34.** Write a TCP client that sends a message.
- **35.** Exchange multiple lines and close sockets safely.
- **36.** Create a database and books table with primary key.
- **37.** Connect using JDBC and report success.
- **38.** Insert a book using PreparedStatement.
- **39.** Insert multiple books in a batch.
- **40.** List books using ResultSet.
- **41.** Search book by ID using a parameterized query.
- **42.** Update a book record.
- **43.** Delete a book after confirmation.

- ■ **44.** Create a members table with suitable key.
- ■ **45.** Register a validated member.
- ■ **46.** Create issue/return table with foreign keys.
- ■ **47.** Implement issue and prevent unavailable stock issue.
- ■ **48.** Implement return and update availability.
- ■ **49.** Store issue and due dates using SQL date types.
- ■ **50.** Calculate overdue days and configurable fine.
- ■ **51.** Use a transaction for multi-step issue.
- ■ **52.** Roll back when a transaction step fails.
- ■ **53.** Use try-with-resources for JDBC.
- ■ **54.** Separate database access into DAO.
- ■ **55.** Create a service for library rules.
- ■ **56.** Validate IDs and required text.
- ■ **57.** Display duplicate/missing-record messages.
- ■ **58.** Search books by title keyword.
- ■ **59.** List currently issued books.
- ■ **60.** List overdue issues.
- ■ **61.** Create a menu-driven library app.
- ■ **62.** Keep SQL errors useful without exposing credentials.
- ■ **63.** Move database configuration out of source.
- ■ **64.** Test CRUD and issue/return with sample data.
- ■ **65.** Write README with schema, setup, features, and run steps.

Monthly total: 65 problems. Mark a problem complete only after testing and explaining your approach.

Month 5 — DSA and SQL

Days 113–140 | Complexity, algorithms, data structures, graph/tree basics, SQL

- ■ **1.** Estimate complexity of one loop over N.
- ■ **2.** Estimate complexity of nested loops over N.
- ■ **3.** Compare linear and binary search; state sorted-input requirement.
- ■ **4.** Implement linear search returning index or -1.
- ■ **5.** Implement iterative binary search.
- ■ **6.** Implement recursive binary search.
- ■ **7.** Implement bubble sort and count comparisons.
- ■ **8.** Implement selection sort.
- ■ **9.** Implement insertion sort.
- ■ **10.** Implement merge sort.
- ■ **11.** Implement quicksort and test edge cases.
- ■ **12.** Find second-largest distinct array value.
- ■ **13.** Move zeros to end preserving nonzero order.
- ■ **14.** Rotate array right by K.
- ■ **15.** Use two pointers for target pair in sorted array.
- ■ **16.** Use sliding window for maximum fixed-size subarray sum.
- ■ **17.** Build prefix sums for range queries.
- ■ **18.** Find most frequent element using frequency map.
- ■ **19.** Implement singly linked list append and display.
- ■ **20.** Insert linked-list node at head and position.
- ■ **21.** Delete a linked-list node by value.
- ■ **22.** Search linked list.
- ■ **23.** Reverse linked list iteratively.
- ■ **24.** Find middle node using slow/fast pointers.
- ■ **25.** Detect a linked-list cycle.
- ■ **26.** Implement doubly linked list traversal both ways.
- ■ **27.** Implement stack using array.
- ■ **28.** Implement stack using linked list.
- ■ **29.** Check balanced parentheses using stack.
- ■ **30.** Evaluate postfix expression with integer operands.
- ■ **31.** Implement array queue.
- ■ **32.** Implement circular queue.
- ■ **33.** Use Deque to test a palindrome sequence.
- ■ **34.** Count frequencies using HashMap.
- ■ **35.** Find first non-repeating string character.
- ■ **36.** Create binary tree and preorder traversal.
- ■ **37.** Implement inorder and postorder traversals.
- ■ **38.** Find binary tree height.
- ■ **39.** Count leaf nodes.
- ■ **40.** Implement BST insertion and search.
- ■ **41.** Find BST minimum and maximum.
- ■ **42.** Implement level-order traversal.
- ■ **43.** Find K largest values using PriorityQueue.

- ■ **44.** Represent graph using adjacency list.
- ■ **45.** Perform BFS from a start vertex.
- ■ **46.** Perform recursive DFS.
- ■ **47.** Count connected components.
- ■ **48.** Check whether a path exists between vertices.
- ■ **49.** Solve a small maze using backtracking.
- ■ **50.** Generate all subsets of a small set.
- ■ **51.** Solve a small climbing-stairs or coin-change DP problem.
- ■ **52.** Compare greedy and exhaustive approaches on a small case.
- ■ **53.** Design Student and Course tables with primary keys.
- ■ **54.** Insert sample rows into both tables.
- ■ **55.** Select using WHERE, ORDER BY, LIMIT.
- ■ **56.** Update a student department and verify.
- ■ **57.** Delete a test row respecting foreign keys.
- ■ **58.** Query COUNT, AVG, MIN, MAX.
- ■ **59.** Group students by department.
- ■ **60.** Join students, enrollments, and courses.
- ■ **61.** Find students without enrollments using an outer join pattern.
- ■ **62.** Find students above average using a subquery.
- ■ **63.** Create an index and explain its use.
- ■ **64.** Use Java JDBC to print a query report.
- ■ **65.** Create a DSA repository with approach, complexity, and tests.
- ■ **66.** Complete a timed five-problem set and review mistakes.
- ■ **67.** Refactor one solution for readability and edge cases.
- ■ **68.** Write a month-end strengths/weaknesses review.
- ■ **69.** Create SQL schema, sample data, and query script.
- ■ **70.** Explain Big-O and one data structure as interview practice.

Monthly total: 70 problems. Mark a problem complete only after testing and explaining your approach.

Month 6 — Java Application Development

Days 141–168 | Git, Maven, JUnit, HTTP/REST, Spring Boot, persistence, testing, capstone

- [ ] **1.** Create a Git repository and initial commit.
- [ ] **2.** Create a feature branch, change code, and merge it.
- [ ] **3.** Write a project README with run instructions.
- [ ] **4.** Create a Maven project and inspect its structure.
- [ ] **5.** Add a dependency and explain pom.xml.
- [ ] **6.** Write a JUnit test for a calculator method.
- [ ] **7.** Test valid and invalid inputs.
- [ ] **8.** Refactor a long method and preserve behavior with tests.
- [ ] **9.** Create a JSON example and identify fields/types.
- [ ] **10.** Explain HTTP GET, POST, PUT, PATCH, DELETE with examples.
- [ ] **11.** Design REST paths for books and members.
- [ ] **12.** Create and run a Spring Boot app with a basic endpoint.
- [ ] **13.** Create a controller GET endpoint.
- [ ] **14.** Use path variables and request parameters.
- [ ] **15.** Create a Book creation DTO.
- [ ] **16.** Implement POST endpoint accepting request body.
- [ ] **17.** Implement GET endpoint returning books.
- [ ] **18.** Implement GET-by-ID with not-found handling.
- [ ] **19.** Implement update endpoint.
- [ ] **20.** Implement delete endpoint with suitable response.
- [ ] **21.** Validate required title and nonnegative quantity.
- [ ] **22.** Create global API exception handling.
- [ ] **23.** Separate controller and service responsibilities.
- [ ] **24.** Configure database using environment-based settings.
- [ ] **25.** Create JPA entity and identifier.
- [ ] **26.** Create repository interface and persistence operation.
- [ ] **27.** Implement database-backed CRUD.
- [ ] **28.** Add search endpoint by title or category.
- [ ] **29.** Write service-layer unit tests.
- [ ] **30.** Test endpoints with an HTTP client and save examples.
- [ ] **31.** Choose capstone domain: library, task tracker, or student records.
- [ ] **32.** Write user stories and limit first-version scope.
- [ ] **33.** Sketch entities and schema.
- [ ] **34.** Implement entity and repository classes.
- [ ] **35.** Implement create and list workflows.
- [ ] **36.** Implement get-by-ID and missing-record behavior.
- [ ] **37.** Implement update and delete.
- [ ] **38.** Add validation and consistent error response.
- [ ] **39.** Add pagination or filtering if feasible.
- [ ] **40.** Test service business rules.
- [ ] **41.** Test API success and failure cases.
- [ ] **42.** Use environment variables for secrets and DB config.
- [ ] **43.** Add seed data or setup steps.

- ■ **44.** Review API status codes and response shapes.
- ■ **45.** Check duplicate/invalid record handling.
- ■ **46.** Run from a clean checkout using README.
- ■ **47.** Fix bugs and remove unused code.
- ■ **48.** Prepare a short demo of main workflows.
- ■ **49.** Write complete README with prerequisites and commands.
- ■ **50.** Document endpoints with request/response examples.
- ■ **51.** Add database schema or setup script.
- ■ **52.** Improve validation messages.
- ■ **53.** Review exception handling for predictable failures.
- ■ **54.** Verify credentials are not committed.
- ■ **55.** Run test suite and record results.
- ■ **56.** Use descriptive Git commits.
- ■ **57.** Create a release tag or version label.
- ■ **58.** Prepare a two-minute project explanation.
- ■ **59.** Explain controller-service-repository flow.
- ■ **60.** Explain HTTP request-to-database-to-response flow.
- ■ **61.** List three limitations and future improvements.
- ■ **62.** Revise Core Java with a mixed quiz.
- ■ **63.** Revisit five DSA problems previously missed.
- ■ **64.** Plan the next 90 days of Java/backend study.
- ■ **65.** Publish repository and verify README from a fresh environment.

Monthly total: 65 problems. Mark a problem complete only after testing and explaining your approach.

Monthly Progress Record

Month 1 — Assigned: 65 Attempted: _____ Solved independently: _____ Revisit: _____

Month 2 — Assigned: 65 Attempted: _____ Solved independently: _____ Revisit: _____

Month 3 — Assigned: 70 Attempted: _____ Solved independently: _____ Revisit: _____

Month 4 — Assigned: 65 Attempted: _____ Solved independently: _____ Revisit: _____

Month 5 — Assigned: 70 Attempted: _____ Solved independently: _____ Revisit: _____

Month 6 — Assigned: 65 Attempted: _____ Solved independently: _____ Revisit: _____

At each month end, record your strongest topics, topics to revisit, one debugging lesson, and the next improvement for your project.